

1. Why Do We Need Async & Parallel Programming?

Modern applications perform **time-consuming operations** such as:

- Database queries
- API / network calls
- File operations
- Heavy computations

At the same time, **modern CPUs have multiple cores** that can execute tasks in parallel.

Problems with single-thread execution

Problem	Effect
Long-running calls block the thread	UI freezes / App not responsive
One thread uses one CPU core	CPU under-utilized

2. Two Solutions in .NET

Goal	Solution	Meaning
Avoid UI freezing	Async/Await	Non-blocking operations
Use multiple CPU cores	Task Parallel Library (TPL)	Run work in parallel

Key Distinction

- **Asynchronous programming = responsiveness**
- **Parallel programming = performance**

They solve **different problems**, but can work together.

Async Programming — Core Concepts

3. async Keyword

Marks a method as asynchronous.

Allowed return types:

- Task
- Task<T>
- void — only for event handlers (avoid otherwise)

4. await Keyword

await tells the compiler to:

- ✓ pause execution
- ✓ **without blocking the thread**
- ✓ resume when the task finishes

So UI remains responsive.

Important Rule

If an async method does **not include await**, it executes synchronously.

5. wait vs await (Very Important)

wait	await
Blocks calling thread	Does NOT block thread
UI freezes	UI stays responsive
Old approach	Modern approach

6. Thread.Sleep vs Task.Delay

Thread.Sleep()	Task.Delay()
Blocks thread	Non-blocking
Freezes UI	UI stays responsive
Hard to cancel	Supports cancellation
Not async-friendly	Async-friendly

✓ **Always prefer Task.Delay() in async code.**

7. When To Use Async Programming

Use async when work is **I/O-bound**, such as:

- File I/O
- API calls
- Network operations
- Database calls

Goal = **do not block the thread while waiting**

Parallel vs Asynchronous — Final Clarity

Concept	Purpose	Example
Asynchronous	Non-blocking	Waiting for database
Parallel	Execute many tasks at once	Processing large dataset

Memory Key

Async = *don't block me*

Parallel = *use all cores*



1. What is the Task Parallel Library (TPL)?

The **Task Parallel Library (TPL)** is a .NET framework introduced in **.NET 4.0** to simplify multithreading and parallel execution.

TPL provides

- ✓ Easier coding than raw threads
- ✓ Automatic CPU core utilization
- ✓ Task-based execution model
- ✓ Built-in exception & cancellation support
- ✓ Uses the thread pool internally

Namespace: System.Threading.Tasks

Core Principle

A Task represents a unit of work that may run asynchronously or in parallel.

2. Task vs Thread (Very Important)

Feature	Task	Thread
Level	High-level abstraction	Low-level
Scheduling	Uses Thread Pool	Manual
Async support	Yes	No
Return values	Built-in (Task<T>)	Complex
Cancellation	Supported	Manual
Exception handling	Structured	Manual
Resource usage	Light	Heavier

✓ Modern .NET recommends Task over Thread

3. Types of Tasks

Task Type	Meaning	Use Case
Task	Work with no return value	Logging, sending emails
Task<T>	Work with a return value	Fetching data, calculations

Examples:

- Task<int> returns an integer
- Task returns nothing

4. Creating & Running Tasks

There are **three standard ways**:

Method	Description	Recommendation
Task.Run()	Creates & starts task immediately	Preferred
Task.Factory.StartNew()	Advanced scheduling scenarios	Rare
new Task(...).Start()	Create then manually start	Least preferred

✓ Exam-style answer → prefer **Task.Run()**

5. Task Lifecycle & Status

A Task can be in one of these states:

- Created
- Running
- RanToCompletion
- Canceled
- Faulted (exception occurred)

Useful for **debugging & diagnostics**.

6. Waiting for Tasks to Finish

Method	Purpose
task.Wait()	Wait for single task
Task.WaitAll()	Wait for multiple tasks
Task.WaitAny()	Continue when first finishes

Reasons to wait:

- ✓ Get result
 - ✓ Handle exceptions
 - ✓ Prevent early program exit
-

7. Tasks Returning Values

If return is required → use **Task<T>**.

Retrieve result via:

- ✓ await task (**preferred — non-blocking**)
 - ✗ task.Result (**blocking — avoid in UI apps**)
-

8. When To Use Task.Run()

Use for **CPU-bound work**, such as:

- ✓ Processing
- ✓ Computation
- ✓ Data transformation

Avoid using Task.Run for UI animation, trivial work, or deterministic timing.

9. Thread.Sleep vs Task.Delay (Context Reminder)

Thread.Sleep()	Task.Delay()
Blocks thread	Non-blocking
Freezes UI	UI responsive
No cancellation	Supports cancel
Old-style	Async-friendly

- ✓ Use Task.Delay in async code
-

Parallel Programming with TPL

10. Parallel Class

The **Parallel class** allows **automatic parallel execution** of independent operations.

Core Methods

Method	Purpose
Parallel.Invoke()	Run multiple actions simultaneously
Parallel.For()	Parallel numeric loops
Parallel.ForEach()	Parallel iteration over collections

Use When Work Is

- ✓ CPU-intensive
 - ✓ Independent (no shared state issues)
-

11. Parallel Programming vs Async Programming — Final Distinction

Parallel Programming	Async Programming
Uses multiple CPU cores	Non-blocking I/O
Improves performance	Improves responsiveness
Best for computations	Best for DB/API/File calls

PLINQ — Parallel LINQ

12. What is PLINQ?

PLINQ enables LINQ queries to **run in parallel automatically** using:

`.AsParallel()`

Use When

- ✓ Work is CPU-heavy
- ✓ Ordering does not matter (unless enforced)
- ✓ Data is thread-safe

Task & Parallel Execution — Best Practices

13. Prefer Task over Thread because

- ✓ Easier to use
- ✓ Uses thread pool
- ✓ Built-in exception handling
- ✓ Return values supported
- ✓ Cancellation available

14. Prefer await over Result because

- ✓ await does NOT block
- ✗ .Result blocks and may deadlock

15. Prefer Task.Delay over Thread.Sleep because

- ✓ Delay does not block
 - ✓ Supports async patterns
-

1. What is LINQ?

LINQ (Language Integrated Query) provides a **single, consistent query syntax** for working with:

- ✓ In-memory collections
- ✓ Objects
- ✓ XML
- ✓ Databases
- ✓ Files

Benefits

- Strongly-typed (compile-time safety)
- IntelliSense support
- Cleaner & more readable code
- Same syntax across data sources

Introduced in **.NET 3.5**.

2. Language Features That Enable LINQ

LINQ is made possible by:

- **var** (implicit typing)
- **Lambda expressions**
- **Extension methods**
- **Anonymous types**
- **Object initializer syntax**

3. Extension Methods — Foundation of LINQ

Extension methods allow adding new methods to existing types **without modifying the type**.

Rules

- Method must be static
- Inside a static class
- First parameter starts with `this`

Example pattern:

```
static Return Type MethodName(this Type t)
```

LINQ operators such as `Where()`, `Select()`, `Count()`, etc. are **extension methods**.

4. LINQ Syntax Styles

A. Query Syntax (SQL-like)

from x in collection

where condition

select x

B. Method Syntax (Lambda Style)

collection.Where(...).Select(...)

✓ Both produce identical results.

5. Deferred Execution (Very Important)

Most LINQ queries are **not executed immediately**.

They execute **only when iterated**, for example via:

- foreach
- ToList()
- ToArray()

Meaning:

✓ If data changes before query is executed, results reflect the latest state.

6. Immediate Execution

Occurs when you explicitly materialize results using:

- ToList()
 - ToArray()
 - ToDictionary()
-

7. Anonymous Types

Used to return **selected fields only** without defining a class.

Example:

```
select new { name = p.Name, desc = p.Description }
```

8. Core LINQ Methods (Must-Know)

Method	Purpose
Where()	Filter elements
Select()	Transform / project data
OrderBy()	Sort ascending
OrderByDescending()	Sort descending
Count()	Count items
Max()	Largest value
Min()	Smallest value
Average()	Mean
First()	First match — throws if none
FirstOrDefault()	First match or default
Single()	Exactly one result expected — throws if not
SingleOrDefault()	Zero or one expected — throws if more

9. Examples with Arrays

int[] a = {5,8,9,2,15,8,9,2,1};

Queries & meanings:

Query	Meaning
a.Where(n <= 5)	Numbers ≤ 5
a.Where(n % 2 == 0)	Even numbers
a.Where(n % 2 != 0)	Odd numbers
a.Where(n >= 8)	Numbers ≥ 8

10. LINQ with Custom Class (Product Example)

Assume:

ProductInfo { Name, Description, NumberInStock }

Example Use-Cases

Goal	Query
Select all objects	items.Select(p => p)
Select only names	items.Select(p => p.Name)
Anonymous projection	items.Select(p => new { p.Name, p.Description })
Stock = 100	items.Where(p => p.NumberInStock == 100)
Name contains "s", sorted	items.Where(p => p.Name.Contains("s")).OrderBy(p => p.Name)
Count items with stock < 100	items.Count(p => p.NumberInStock < 100)
Max / Min / Average stock	Max(), Min(), Average()

11. Repository Pattern & LINQ

A repository typically provides:

- Display()
- Add()
- Remove(id)
- GetById(id)
- GetByName(name)
- Update(id)



LINQ simplifies internal queries — making the repository cleaner.

12. Execution Order for Model Binding Reminder

(Not repeated — only context clarity)

LINQ **does not execute until iterated** — key takeaway.

1. Why Do We Need Joins in LINQ?

Very often, data lives in **multiple related collections**, similar to relational databases.

Example scenario:

- Employee → contains DeptId
- Department → contains DeptId

To **combine data logically**, we use **JOIN** — just like SQL.

2. LINQ JOIN — Core Concept

A **Join** combines two collections based on a **matching key**.

Equivalent to SQL INNER JOIN

✓ Only matching records are returned.

3. Syntax of LINQ Join

General structure:

from a in collectionA

join b in collectionB

on a.Key equals b.Key

select ...

✓ Note that **equals** is required — not ==.

4. Example — Employee & Department Join

Assume

Employee { EmpId, EmpName, DeptId }

Department { DeptId, DeptName }

Join Query

from e in employees

join d in departments

on e.DeptId equals d.DeptId

select new { e.EmpName, d.DeptName }z

Meaning

- ✓ Employees are matched with their department names
- ✓ Only employees with valid DeptId appear

5. Important Join Keywords

Keyword	Purpose
join	Specifies collection to join
on	Defines matching key
equals	Required comparison keyword
select	Defines output

GROUPBY in LINQ

6. What is GroupBy?

GroupBy organizes data **based on a key**, producing **groups of records**.

Return Type

IEnumerable<IGrouping<TKey, TElement>>

Each group contains:

- ✓ Key
- ✓ Collection of elements belonging to that key

7. Why GroupBy Is Useful

Used heavily in:

- ✓ Reporting
- ✓ Analytics
- ✓ Statistics dashboards
- ✓ Logical grouping

Often used with:

- Count()
 - Sum()
 - Average()
-

8. Employee GroupBy Example

Group employees **by department**:

```
var result = employees.GroupBy(e => e.Department);
```

Output Conceptually

HR → Raj, Raja, Ganesh

MKT → Kavita, Vidya, Mohan

ADV → Rajani

9. Accessing Group Results

Each group has:

- .Key
- Iteration over elements

Example:

```
foreach (var g in result)
{
    Console.WriteLine(g.Key);
    foreach (var emp in g)
        Console.WriteLine(emp.Name);
}
```



PLINQ — Parallel LINQ (Integration Topic)

10. What is PLINQ?

PLINQ runs LINQ queries in **parallel across CPU cores** using:

```
.AsParallel()
```

11. When To Use PLINQ

Use when:

- ✓ Data volume is large
 - ✓ Work per element is CPU-heavy
 - ✓ Tasks are independent
 - ✓ Order doesn't matter (unless you enforce order)
-

12. Example

```
var result = data
    .AsParallel()
    .Where(x => Compute(x) > 0)
    .ToList();
```

✓ Execution automatically spreads across CPU cores.

Repository Pattern — Where LINQ Fits

13. Repository Responsibilities

Typical repository operations:

- Display()
- Add()
- Remove(id)
- GetById(id)
- GetByName(name)
- Update(id)

LINQ simplifies filtering logic internally

Example:

```
return employees.Where(e => e.Id == id).FirstOrDefault();
```

- ✓ Cleaner
- ✓ More readable
- ✓ Maintains business logic consistency

JOIN vs GROUPBY — Exam-Oriented Clarity

14. Difference Table

Feature	JOIN	GROUPBY
Purpose	Combine records from different collections	Organize records into logical groups
Output	Flat combined records	Group objects with key + list
Keywords	join, on, equals	GroupBy
Typical Use	Employee + Department	Count employees per department

15. Full Topic — High-Value Golden Summary (Must-Revise Section)

This summary combines **TPL + Async + LINQ** knowledge into one revision block.

Async & Await

- Prevent UI blocking
 - await does **not block** calling thread
 - wait **does block**
 - Use Task.Delay() instead of Thread.Sleep()
 - Async is ideal for **I/O-bound** operations
-

Task Parallel Library (TPL)

- Task is preferred over Thread
 - Task<T> returns values
 - Task.Run() is the recommended way to start background tasks
 - Task lifecycle states:
Created / Running / RanToCompletion / Canceled / Faulted
 - Use Wait, WaitAll, WaitAny when blocking is necessary
 - **Parallel class**
 - Parallel.Invoke
 - Parallel.For
 - Parallel.ForEach
 - **Parallel = performance**
 - **Async = responsiveness**
-

PLINQ

- Enables parallel execution of LINQ queries
 - Use .AsParallel()
 - Best for CPU-bound workloads
-

LINQ Core

- Unified query syntax for collections, XML, DB, files
- Supported by:

- Lambdas
 - Extension methods
 - Anonymous types
 - **Deferred execution** until enumerated
 - **Immediate execution** with ToList, ToArray, ToDictionary
-

Most-Used LINQ Methods

- Where → filter
 - Select → project/transform
 - OrderBy / OrderByDescending
 - Count, Max, Min, Average
 - First → throws if none
 - FirstOrDefault → safe default
 - Single / SingleOrDefault → ensure exactly one record
-

Join & GroupBy

- **Join** → combine related collections on a key
 - **GroupBy** → group items under a key
 - Both widely used in real-world apps & interviews
-